

Zalord — Hệ thống Chat Microservices

Kiến trúc & Triển khai

Nhóm 17

Trường Đại học Công nghệ Thông tin — ĐHQG TP.HCM
GVHD: TS. Nguyễn Duy Khánh

Tháng 6, 2026

Nội dung trình bày

- 1 Tổng quan hệ thống
- 2 Domain-Driven Design
- 3 Nguyên lý thiết kế Microservices
- 4 Giao tiếp giữa các dịch vụ
- 5 Thiết kế & Quản lý cơ sở dữ liệu
- 6 Triển khai Docker & Kubernetes
- 7 API Gateway
- 8 Authentication & Authorization
- 9 Logging, Monitoring, Tracing
- 10 CI/CD & Triển khai
- 11 Kết luận

Zalord là gì?

- Ứng dụng chat real-time kiến trúc **Microservices**
- **7 services** độc lập, polyglot Java / Go
- Hỗ trợ: nhắn tin 1-1, nhóm, file đính kèm, real-time presence
- Frontend: React 19 + TypeScript + Vite + WebSocket

Tech stack tổng quan

- **Gateway:** Kong 3.7
- **Messaging:** RabbitMQ / Kafka (switchable)
- **Storage:** PostgreSQL ×6, Redis, MinIO (S3)
- **Observability:** Prometheus + Grafana
- **Deploy:** Docker Compose (local) / k3s VPS (stag) / GKE (prod)
- **CI/CD:** GitHub Actions → Docker Hub → GKE / k3s

Danh sách Microservices

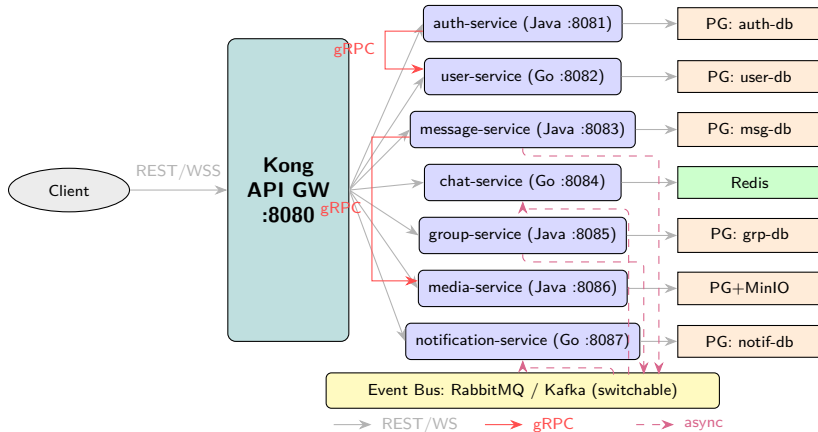
Service	Lang	Port
auth-service	Java/Spring	8081
user-service	Go	8082
message-service	Java/Spring	8083
chat-service	Go	8084
group-service	Java/Spring	8085
media-service	Java/Spring	8086
notification-service	Go	8087

gRPC Internal (không qua Kong)

auth → user (CreateProfile :9082)

message → media (ValidateAttachments :9086)

Kiến trúc tổng quan



Code Organization

Tại sao chọn Layered thay vì Clean Architecture hay Vertical Slice?

Tiêu chí	Layered (Đang dùng)	Clean / Hexagonal	Vertical Slice (VSA)
Số file / tính năng	3 file (Controller, Service, Repo)	5-7 file (Port, Adapter, Use-case, Entity, DTO)	3-4 file (Request, Response, Handler inline)
Share Dependencies (VD: Config, Repo)	Dễ dàng tiêm 1 lần vào chung class Service	Dễ dàng tiêm vào chung Use case class	Khó — mỗi slice bị cô lập, phải tự tiêm lại từ đầu
Độ tương thích với Framework	Tuyệt đối (Hưởng lợi từ Spring annotations)	Xung đột (Domain không được chứa thư viện Spring/JPA)	Kém (Spring không hỗ trợ MediatR built-in như .NET)
Độ phức tạp & Duplication	Thấp, code base đơn giản, tập trung	Boilerplate quá cao so với quy mô dự án 3 tháng	Trùng lặp code nhiều (VD: Outbox pattern phải copy)

Kết luận cho bài toán thực tế của Zalord

Team 3 người, thời hạn 3 tháng, service nhỏ (~8-10 endpoints).

→ **Layered Architecture** mang lại tốc độ code nhanh nhất, tận dụng tối đa tiện ích của Spring Boot, không bị "over-engineering".

Tại sao không dùng Modulithic? (Req 1)

Lý do chọn Microservices

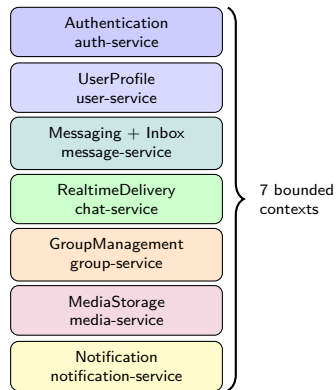
- **Polyglot language:** chat-service dùng Go (goroutines) cho WebSocket fan-out hiệu năng cao; business logic dùng Java Spring Boot
- **Independent scaling:** chat-service cần scale riêng khi traffic tăng, không kéo theo auth-service
- **Independent deploy:** cập nhật media-service không ảnh hưởng chat real-time
- **Fault isolation:** media-service lỗi không kéo sập toàn bộ hệ thống

Nếu là Modulithic...

- Không thể mix Go + Java trong 1 process
- Scale toàn bộ khi chỉ cần scale 1 module
- Deploy toàn bộ khi chỉ thay đổi 1 module
- 1 memory leak / goroutine leak làm sập tất cả

Phân chia theo DDD — Bounded Contexts (Req 2)

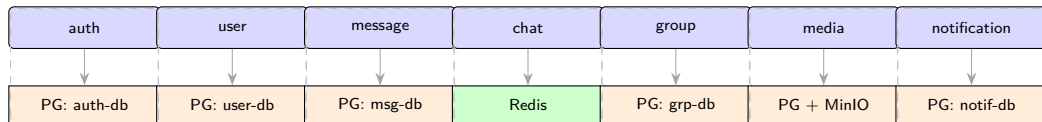
Service	Bounded Context	Aggregate Root
auth-service	Authentication	Account, Session
user-service	UserProfile	UserProfile
message-service	Messaging + Inbox	Message, ConversationView
chat-service	RealtimeDelivery	Connection
group-service	GroupManagement	Group
media-service	MediaStorage	MediaItem
notification-service	Notification	Notification



DB per Bounded Context (Req 2)

Nguyên tắc phân chia

- Mỗi service có **1 bounded context** độc lập
- Không share entity/table giữa các service
- Giao tiếp qua **API / Event** (không gọi thẳng DB)
- conversation.id được tái sử dụng làm group.id (shared key, không shared schema)



Mỗi service chỉ truy cập database của chính mình — không share schema

Redis (shared): auth sessions, chat presence/pub-sub, message membership cache

MinIO (shared): avatars, attachments

Single Responsibility Principle (Req 3)

SRP — Mỗi service 1 trách nhiệm rõ ràng

- **chat-service**: CHỈ quản lý WebSocket connections + fan-out messages → không write business DB, không có business logic
- **media-service**: CHỈ quản lý upload lifecycle (generate presigned URL → MinIO → finalize metadata)
- **notification-service**: CHỈ consume events + persist + serve notification list
- **auth-service**: CHỈ authentication (issue/validate token, session management)

High Cohesion & Low Coupling (Req 3)

High Cohesion — Low Coupling

- **High Cohesion:** dữ liệu và logic cùng domain nằm trong 1 service, không scatter
- **Low Coupling:** chỉ **2 synchronous dependencies** (gRPC):
 - auth → user (registration)
 - message → media (validate attachments)
- Mọi giao tiếp còn lại: **async event bus**
- Downstream services không parse JWT — trust header từ Kong

Execution Pipeline

Request → Kong (validate) → Service → DB / Event Bus

Không có cross-service synchronous chain dài hơn 2 hops

Sync vs Async — Khi nào dùng gì? (Req 4)

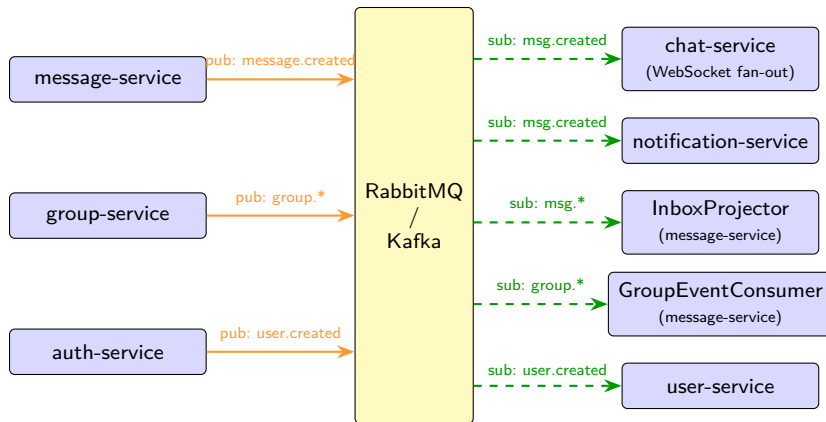
Synchronous — gRPC (2 trường hợp)

- **auth** → **user-service** (CreateProfile):
 - Cần response ngay để rollback nếu profile creation fail
 - Deadline 5 giây, circuit breaker Resilience4j
- **message** → **media-service** (ValidateAttachments):
 - Validate media IDs trước khi persist message
 - Deadline 3 giây (hot path user-facing)
- Dùng gRPC vì: type-safe contract (Protobuf), binary encoding nhanh hơn REST, không qua Kong (internal network)

Asynchronous — Event Bus (Kafka / RabbitMQ)

- **message.created** → chat-service (WebSocket fan-out), notification-service, InboxProjector
- **group.*** (created/member-added/removed/updated) → message-service (project Conversation)
- **user.created** → user-service
- Dùng async vì: fan-out 1-to-many, producers không cần chờ consumers, failure isolation
- **Switchable**: EVENT_BUS=rabbitmq hoặc kafka — cùng interface EventBus (Factory pattern)

Event-Driven Architecture — Message Flow (Req 4)



Benchmark: Kafka vs RabbitMQ

Kết quả Benchmark

- **Kafka:** Throughput cao hơn (hàng trăm ngàn msg/s), phù hợp cho log aggregation và event sourcing.
- **RabbitMQ:** Độ trễ (latency) thấp hơn ở mức tải trung bình, hỗ trợ routing linh hoạt (topic exchange, dead-letter queue).

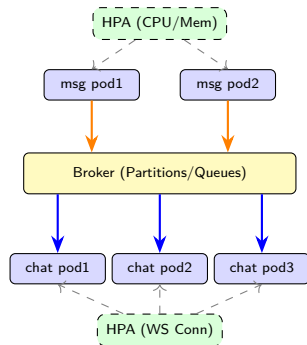
[PLACEHOLDER — biểu đồ benchmark]

Biểu đồ so sánh Throughput & Latency giữa Kafka và RabbitMQ

Scaling cho luồng gửi Message

Chiến lược Scaling

- **Producer (message-service):** Scale theo CPU/Memory (HPA) vì việc lưu DB và publish event đòi hỏi xử lý nhanh.
- **Consumer (chat-service):** Scale theo số lượng kết nối WebSocket đang mở, giới hạn số lượng connection per pod.
- **Broker:** Tăng số lượng Partition (Kafka) hoặc chia nhỏ Queue (RabbitMQ) để scale việc tiêu thụ event.



Thiết kế Endpoint & Swagger (Req 5)

REST API — Naming Convention

- Base path: `/api/v1/{resource}` (plural noun)
- POST `/api/v1/auth/register`
- GET `/api/v1/users`
- GET `/api/v1/conversations`
- POST `/api/v1/messages`
- POST `/api/v1/groups/{id}/members`
- DELETE `/api/v1/groups/{id}/members/{uid}`
- POST `/api/v1/media/upload-url`
- WebSocket: `/ws/chat?token={jwt}`

Error Response

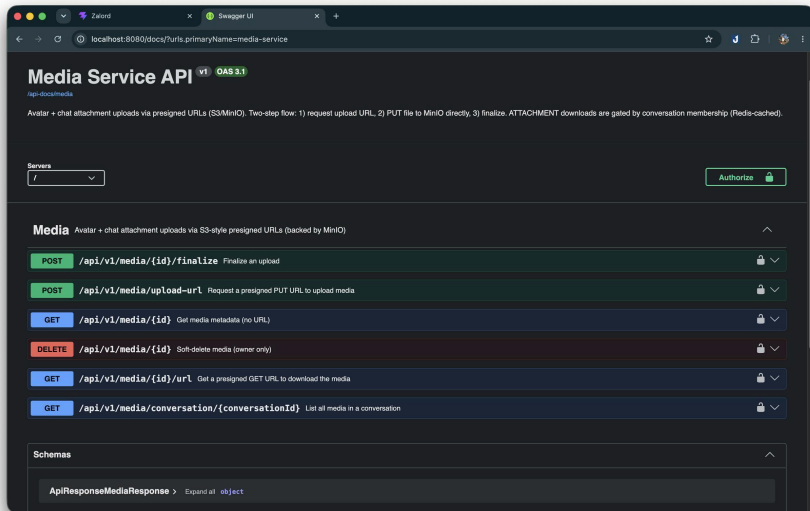
Chuẩn **RFC 7807**
(Platform-agnostic):

```
{"type":...,  
  "status":400,  
  "detail":"...",  
  "title":"..."}
```


Swagger / OpenAPI Aggregation

- Java services: **springdoc-openapi** → /v3/api-docs
- Go services: **swaggo** → /v3/api-docs
- Kong route /api-docs/{svc} → strip_path: true → forward đến service /v3/api-docs
- **Swagger UI** container tại /docs aggregate tất cả

Swagger / OpenAPI Aggregation



DB per Service & Consistency (Req 6)

DB per Service

- 6 PostgreSQL instances độc lập + Redis + MinIO
- Không service nào truy cập trực tiếp DB của service khác
- **Strong consistency**: trong 1 service (ACID transactions)
- **Eventual consistency**: cross-service qua event bus
 - group member change → message-service projection có độ trễ nhỏ
 - message.created → inbox view update không đồng thời

Consistency cần thiết ở đâu?

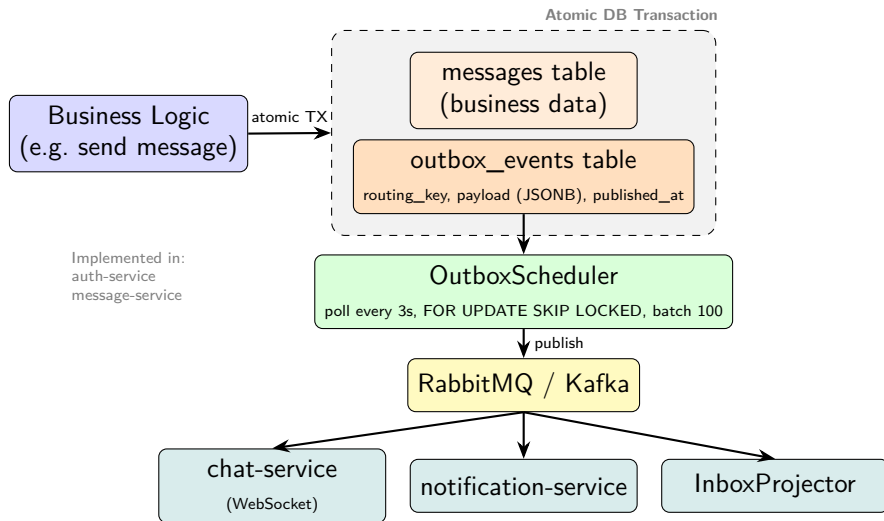
- **Strong**: auth (login/register), message persist (không mất tin nhắn)
- **Eventual**: inbox unread count, notification, presence status

CAP Theorem(Req 6)

CAP Theorem

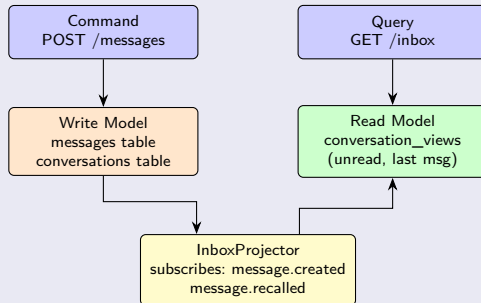
- PostgreSQL: **CP** (Consistency + Partition Tolerance)
- Chấp nhận availability giảm khi partition
- Redis: **AP** — eventual consistency cho presence data

Transactional Outbox Pattern (Req 7)



CQRS trong message-service (Req 7)

CQRS trong message-service

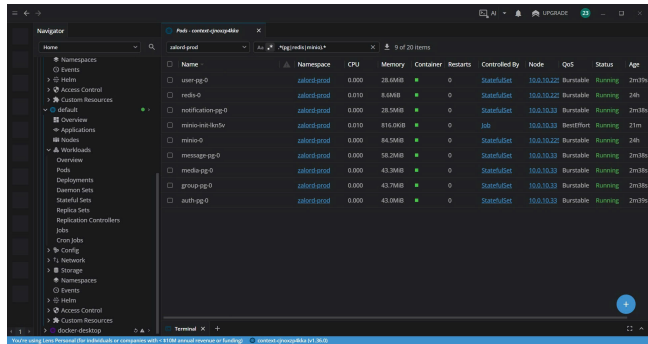


projection có độ trễ nhỏ (eventual consistency)

Polyglot Persistence (Req 7)

Polyglot Persistence

Store	Dùng cho
PostgreSQL ×6	Business data (ACID)
Redis	Sessions, presence, pub/sub
MinIO (S3)	Avatars, attachments



Multi-Stage Dockerfile — Java (auth-service)

```
# Stage 1: Build
FROM eclipse-temurin:21-jdk-jammy AS build
WORKDIR /workspace
COPY mvnw . && COPY .mvn .mvn
COPY pom.xml .
RUN chmod +x mvnw && ./mvnw -B dependency:go-offline
# Layer cache: deps downloaded before source copy
COPY src src
RUN ./mvnw -B -DskipTests clean package \
    && java -Djarmode=layertools -jar target/*.jar \
    extract --destination target/extracted

# Stage 2: Runtime (JRE only, alpine = small)
FROM eclipse-temurin:21-jre-alpine
RUN addgroup -S spring && adduser -S spring -G spring
USER spring:spring
COPY --from=build /workspace/target/extracted/ ./
EXPOSE 8081
```

Multi-Stage Dockerfile — Go (chat-service)

```
# Stage 1: Build
FROM golang:1.26.1-alpine AS builder
WORKDIR /app
COPY go.mod go.sum ./
RUN go mod download          # deps cached separately
COPY . .
RUN CGO_ENABLED=0 go build -o server ./cmd/server

# Stage 2: Runtime (scratch-like, ~8MB)
FROM alpine:latest
WORKDIR /app
COPY --from=builder /app/server .
EXPOSE 8084
CMD ["/server"]
```


Docker Compose

- 20+ containers, 1 network zalord-net
- Named volumes per DB (pgdata_auth, ..., miniodata, redisdata)
- Env vars từ .env file, không hardcode

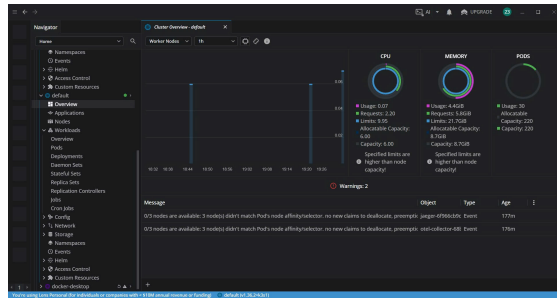
Kustomize Overlays

- Path: deploy/k8s/overlays/dev/
- Namespace: zalord-dev
- **Workloads:** 7 Deployments (services)
- **StatefulSets:** Kafka, RabbitMQ, Redis, PostgreSQL×6, MinIO (kèm Persistent Volumes 1–5Gi)
- **Resources:** req/limit (e.g. auth: 100m/384Mi req)
- **Probes:** TCP liveness, HTTP /health
- **Secrets:** SOPS + Age encryption

```
zsh@zalord-dev:~$ kubectl get pods -n zalord-dev
No resources found in zalord-dev namespace.
zsh@zalord-dev:~$ kubectl get pods -n zalord-staging
NAME                                READY    STATUS    RESTARTS   AGE
auth-pg-0                          1/1      Running   0           18h
auth-service-5d4b4d4d4d-cq7n4      1/1      Running   0           18h
chat-service-577d4d4d4d-r46zh      1/1      Running   4 (3h14m ago) 3h14m
chat-service-577d4d4d4d-tzmxp      1/1      Running   6 (5h0m ago) 5h12m
frontend-0d0b4d4d4d-c45u8         1/1      Running   0           18h
grafana-57b4d4d4d4-7ytrb          1/1      Running   0           18h
group-pg-0                          1/1      Running   0           18h
group-service-9d57d4d4d4-9w5g      1/1      Running   0           18h
jaeger-4d96d4d4d4-cv5fn            0/1      Pending   0           3h2m
jaeger-7fcd4d4d4d-3qg7f            1/1      Running   40 (169m ago) 18h
kafka-0                             1/1      Running   1 (13h ago) 3h1m
media-pg-0                         1/1      Running   0           18h
media-service-5d4d4d4d4d-wfcb       1/1      Running   0           18h
message-pg-0                       1/1      Running   0           3h1m
message-service-3d4d4d4d4d-2b3d8    1/1      Running   0           3h14m
message-service-3d4d4d4d4d-4b4c6    1/1      Running   2 (3h13m ago) 3h14m
notification-pg-0                  1/1      Running   0           18h
notification-service-6d4d4d4d4d-822e 1/1      Running   6 (5h0m ago) 5h12m
notification-service-6d4d4d4d4d-j4g5 1/1      Running   4 (3h14m ago) 3h14m
otel-collector-5d7d4d4d4d-pc48c     1/1      Running   25 (17h ago) 18h
otel-collector-6b4d4d4d4d-b45ch     0/1      Pending   0           3h1m
prometheus-6d5d4d4d4d-4p4mf        1/1      Running   0           18h
rabbitmq-0                         1/1      Running   0           18h
redis-0                            1/1      Running   0           18h
user-pg-0                          1/1      Running   0           18h
user-service-df4d4d4d4d-4b4hr       1/1      Running   3 (18h ago) 18h
zsh@zalord-dev:~$
```

K8s Lens / OpenLens

- Sử dụng **K8s Lens** làm công cụ quản lý cluster trực quan (GUI).
- Dễ dàng theo dõi trạng thái Pods, Nodes, Deployments, và StatefulSets.
- Hỗ trợ xem logs, exec vào container, và chỉnh sửa YAML trực tiếp không cần thao tác dòng lệnh.



Service Discovery (Req 9)

Service Discovery — K8s DNS

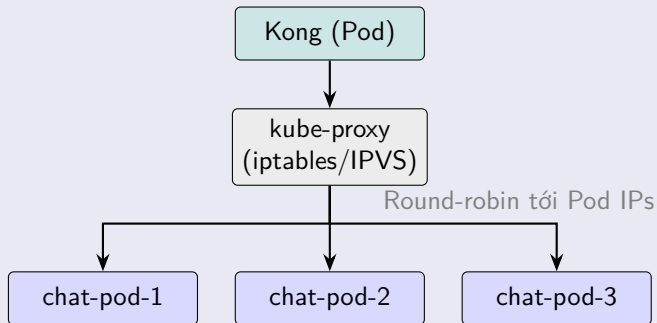
- K8s tự tạo DNS entry cho mỗi ClusterIP Service
- `auth-service.zalord-dev.svc.cluster.local`
- Các service gọi nhau qua tên service nội bộ (e.g. `http://user-service:8082`)
- **KHÔNG cần Eureka hay Consul** — K8s DNS thay thế hoàn toàn

Cấu hình gRPC target

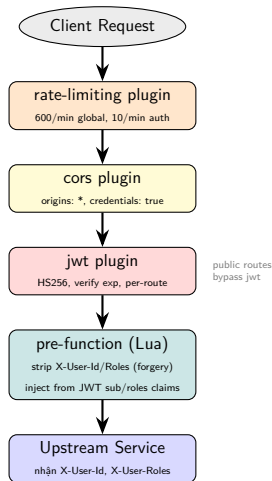
- `ZALORD_USER_SERVICE_GRPC_TARGET=user-service:9082`
- `ZALORD_MEDIA_SERVICE_GRPC_TARGET=media-service:9086`
- Quá trình: DNS resolve → ClusterIP → Pod IP

Load Balancing (Req 9)

Load Balancing — Server-side



Kong API Gateway — Filter Chain (Req 10)



Các tính năng Kong đã cấu hình

- **Rate Limiting:** 600 req/min per IP (global), 10 req/min (auth routes /login, /register)
- **CORS:** global, tất cả origins (dev), credentials enabled
- **JWT Filter:** validate HS256, kiểm tra exp claim
- **Identity Injection:** Lua pre-function inject X-User-Id (từ sub) và X-User-Roles (từ roles array)
- **Prometheus:** metrics tại admin :8001/metrics

Kong — Route Table (Req 10)

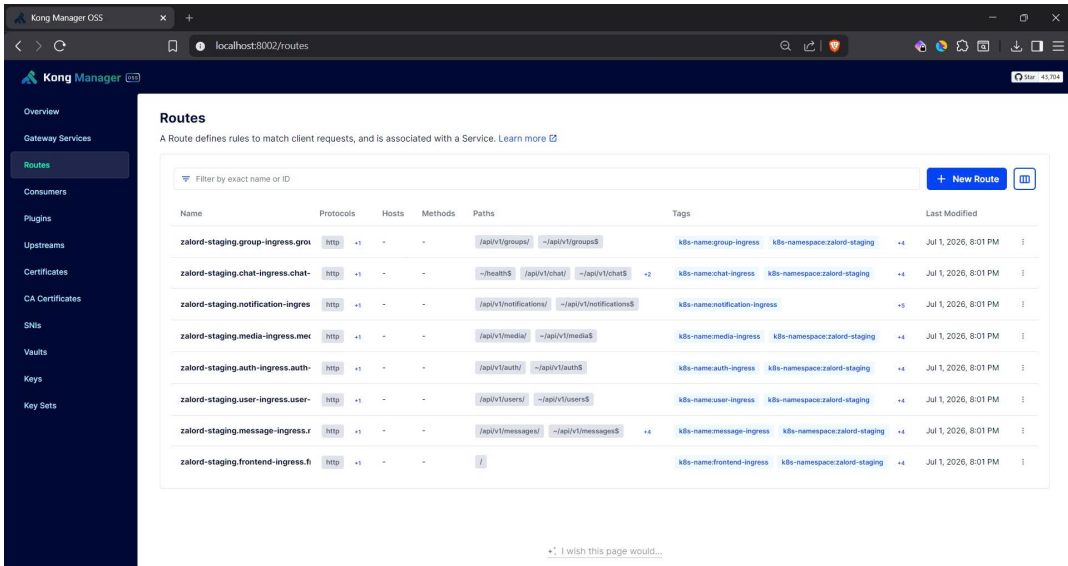
Declarative Config (DB-less)

- Config tại `infra/kong/kong.yml` (version control). **Cùng 1 file** cho Dev & Prod (tránh config drift).
- `__JWT_SECRET__` được thay bằng env var khi start — bảo mật secret.
- **KHÔNG** dùng **K8s Ingress Controller** (`networking.k8s.io/v1`). Đây là **lựa chọn thiết kế** standalone (xem `EXPLANATION.md`).

Route Table

Route	Upstream	JWT
<code>/api/v1/auth/*</code>	<code>auth:8081</code>	×
<code>/api/v1/users</code>	<code>user:8082</code>	✓
<code>/api/v1/messages</code>	<code>message:8083</code>	✓
<code>/ws/chat</code>	<code>chat:8084</code>	✓
<code>/api/v1/groups</code>	<code>group-service:8085</code>	✓
<code>/api/v1/media</code>	<code>media-service:8086</code>	✓
<code>/api/v1/notifications</code>	<code>notification-svc:8087</code>	✓
<code>/docs</code>	<code>swagger-ui:8080</code>	×

Kong API Gateway — Manager UI



Kong Manager OSS

localhost:8002/routes

Kong Manager 0.15.0

Star 43,704

Overview

Gateway Services

Routes

Consumers

Plugins

Upstreams

Certificates

CA Certificates

SNIs

Vaults

Keys

Key Sets

Routes

A Route defines rules to match client requests, and is associated with a Service. [Learn more](#)

Filter by exact name or ID

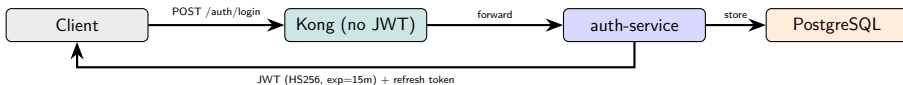
[+ New Route](#)

Name	Protocols	Hosts	Methods	Paths	Tags	Last Modified
zalord-staging.group-ingress.gro	http	+1	-	-	/api/v1/groups/ ~/api/v1/groups\$	k8s-name:group-ingress k8s-namespace:zalord-staging +4 Jul 1, 2026, 8:01 PM
zalord-staging.chat-ingress.chat-	http	+1	-	-	~/health\$ /api/v1/chat/ ~/api/v1/chat\$ +2	k8s-name:chat-ingress k8s-namespace:zalord-staging +4 Jul 1, 2026, 8:01 PM
zalord-staging.notification-ingres	http	+1	-	-	/api/v1/notifications/ ~/api/v1/notifications\$	k8s-name:notification-ingress +5 Jul 1, 2026, 8:01 PM
zalord-staging.media-ingress.mer	http	+1	-	-	/api/v1/media/ ~/api/v1/media\$	k8s-name:media-ingress k8s-namespace:zalord-staging +4 Jul 1, 2026, 8:01 PM
zalord-staging.auth-ingress.auth-	http	+1	-	-	/api/v1/auth/ ~/api/v1/auth\$	k8s-name:auth-ingress k8s-namespace:zalord-staging +4 Jul 1, 2026, 8:01 PM
zalord-staging.user-ingress.user-	http	+1	-	-	/api/v1/users/ ~/api/v1/users\$	k8s-name:user-ingress k8s-namespace:zalord-staging +4 Jul 1, 2026, 8:01 PM
zalord-staging.message-ingress.r	http	+1	-	-	/api/v1/messages/ ~/api/v1/messages\$ +4	k8s-name:message-ingress k8s-namespace:zalord-staging +4 Jul 1, 2026, 8:01 PM
zalord-staging.frontend-ingress.fi	http	+1	-	-	/	k8s-name:frontend-ingress k8s-namespace:zalord-staging +4 Jul 1, 2026, 8:01 PM

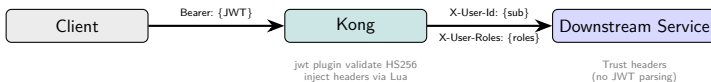
✳️ I wish this page would...

JWT Authentication Flow (Req 11)

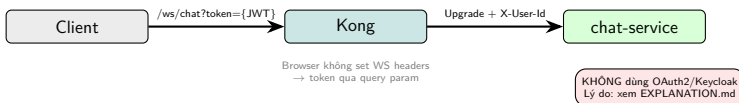
(1) Login



(2) API Call



(3) WebSocket



ConfigMap — Non-sensitive config (Req 12)

ConfigMap — Non-sensitive config

- **Shared:** shared-config.yaml
 - EVENT_BUS=rabbitmq
 - RABBITMQ_HOST=rabbitmq
 - KAFKA_BOOTSTRAP=kafka:9092
 - REDIS_HOST=redis
 - JWT_EXPIRY_MINUTES=15
- **Per-service:** {svc}-config.yaml — database host/port/name, gRPC targets, service-specific config
- Mount as env vars vào Deployment

Tại sao cần ConfigMap?

- Thay đổi config không cần build lại Docker image
- Khác nhau giữa environments (dev/staging/prod) qua Kustomize overlays

Secret — Sensitive data (Req 12)

Secret — Sensitive data (SOPS encrypted)

- **Shared:** DB passwords, JWT_SECRET, MQ credentials
- **Per-service:** database URLs với credentials
- Files: *.secret.enc.yaml — encrypted với **SOPS + Age key**
- Age key lưu trong **GitHub Actions Secret**
- CI/CD decrypt tại deploy time: `sops -d file.enc.yaml`
- Secret encrypted at rest trong Git — an toàn commit public repo

Kong JWT Secret

kong.yml chứa `__JWT_SECRET__` placeholder. Container entrypoint dùng `sed` substitute từ K8s Secret env var. Secret không bao giờ nằm trong file config.

Prometheus & Grafana — Observability (Req 13)

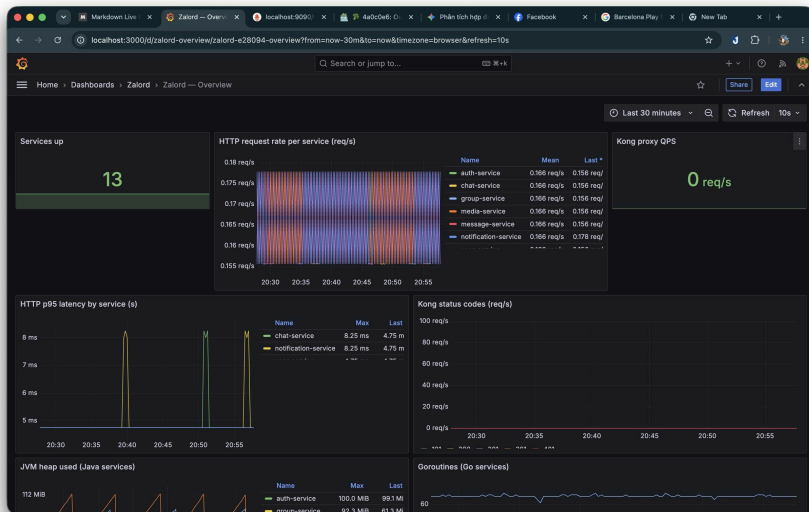
Prometheus Scrape Config

Target	Endpoint
Java Services	/actuator/prometheus
Go Services	/metrics
Kong	kong:8001/metrics
RabbitMQ	rabbitmq:15692/metrics
Kafka	kafka-exporter:9308

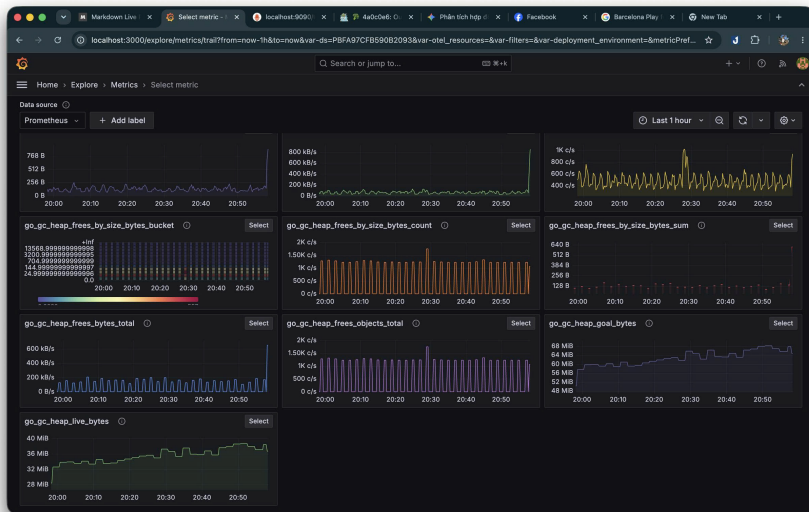
Metrics được track

- HTTP latency, request count, status codes
- Active WebSocket connections (chat)
- Message throughput, queue lag
- JVM heap, GC, Pod resource

Prometheus & Grafana — Dashboard



Prometheus & Grafana — Graph



Grafana Loki

- Thay vì dùng stack ELK nặng nề, hệ thống dùng **Loki** + **Promtail** nhẹ hơn nhiều, tối ưu cho K8s.
- Promtail tự động thu thập stdout của tất cả pods và đẩy lên Loki.
- Truy vấn log tập trung thông qua **Grafana** (LogQL). Không cần login vào từng pod bằng `kubectl logs`.

Centralized Logging

The screenshot displays the Grafana Logs Drilldown interface. The left sidebar contains navigation links: Home, Bookmarks, Starred, Dashboards, Playlists, Snapshots, Library panels, Public dashboards, Explore (selected), Alerting, Alert rules, Contact points, Notification policies, Silences, Active notifications, Settings, Connections, and Administration. The main panel shows a log stream for the 'message-service' with a search bar and filters. The log entries are displayed in a table format, showing timestamps, log levels, and message content. The logs are filtered by 'message-service' and 'InboxProjector'.

Search or jump to...

500
0

19:47:00 19:48:00 19:49:00 19:50:00 19:51:00 19:52:00 19:53:00 19:54:00 19:55:00 19:56:00 19:57:00 19:58:00 19:59:00 20:00:00 20:01:00

unknown Total: 12071 info Total: 6695 warn Total: 6

Search in Log Lines Include Exclude

Logs (5K)

Newest first Oldest first Wrap No wrap Logs Table

```
> 2026-07-01 20:00:07.383 2026-07-01T13:00:07.383Z DEBUG 1 --- [message-service] [a.Spring.bean-1] z.message.service.worker.InboxProjector : Inbox proje
cted for conv1cd3e397-48e8-42fc-9838-8db7fdd7c7ed message=3456b967-7ed9-4c2f-acb3-d2ac8675642d members=2

> 2026-07-01 20:00:07.384 2026-07-01T13:00:07.384Z DEBUG 1 --- [message-service] [a.Spring.bean-1] z.message.service.worker.InboxProjector : Inbox proje
cted for conv1cd3e397-48e8-42fc-9838-8db7fdd7c7ed message=6f83fa78-f328-4ea9-a9d5-1e7d7367d3a members=2

> 2026-07-01 20:00:07.384 2026-07-01T13:00:07.384Z INFO 1 --- [message-service] [nio-8883-exec-8] z.m.service.lmp1.MessageServiceImpl : Message sen
t id=cf022bd-f98f-45fd-b3b3-bfc31588171 conv1cd3e397-48e8-42fc-9838-8db7fdd7c7ed sender=c84a1965-ddab-45ea-a255-142b7543a38e
recipients=1 attachments=0 replyTo=null

> 2026-07-01 20:00:07.385 2026-07-01T13:00:07.385Z DEBUG 1 --- [message-service] [a.Spring.bean-1] z.message.service.worker.InboxProjector : Inbox proje
cted for conv1cd3e397-48e8-42fc-9838-8db7fdd7c7ed message=f4992e1a-0f01-4f55-b809-ec5a9672b2f6 members=2

> 2026-07-01 20:00:07.386 2026-07-01T13:00:07.386Z DEBUG 1 --- [message-service] [a.Spring.bean-1] z.message.service.worker.InboxProjector : Inbox proje
cted for conv1cd3e397-48e8-42fc-9838-8db7fdd7c7ed message=28b37e64-0f9a-4c82-b885-f243f85fe8a4 members=2

> 2026-07-01 20:00:07.387 2026-07-01T13:00:07.387Z DEBUG 1 --- [message-service] [a.Spring.bean-1] z.message.service.worker.InboxProjector : Inbox proje
cted for conv1cd3e397-48e8-42fc-9838-8db7fdd7c7ed message=12d438fa-ca69-4e9b-909e-b828b58faa5b members=2

> 2026-07-01 20:00:07.388 2026-07-01T13:00:07.387Z DEBUG 1 --- [message-service] [a.Spring.bean-1] z.message.service.worker.InboxProjector : Inbox proje
cted for conv1cd3e397-48e8-42fc-9838-8db7fdd7c7ed message=76bea8f5-f498-4fac-83a3-79df53e5cd2f members=2

> 2026-07-01 20:00:07.389 2026-07-01T13:00:07.389Z INFO 1 --- [message-service] [nio-8883-exec-10] z.m.service.lmp1.MessageServiceImpl : Message sen
t id=c780832d-47aa-4b61-82ee-a5f9b63af1d8 conv1cd3e397-48e8-42fc-9838-8db7fdd7c7ed sender=c84a1965-ddab-45ea-a255-142b7543a38e
recipients=1 attachments=0 replyTo=null

> 2026-07-01 20:00:07.314 2026-07-01T13:00:07.314Z INFO 1 --- [message-service] [nio-8883-exec-1] z.m.service.lmp1.MessageServiceImpl : Message sen
t id=66962bda-cab5-4ea8-b1f3-8d68aa52a39 conv1cd3e397-48e8-42fc-9838-8db7fdd7c7ed sender=c84a1965-ddab-45ea-a255-142b7543a38e
recipients=1 attachments=0 replyTo=null

> 2026-07-01 20:00:07.319 2026-07-01T13:00:07.319Z INFO 1 --- [message-service] [nio-8883-exec-9] z.m.service.lmp1.MessageServiceImpl : Message sen
t id=1dc4afdd-38b4-4b88-9b72-b4fd1d6e5f72 conv1cd3e397-48e8-42fc-9838-8db7fdd7c7ed sender=c84a1965-ddab-45ea-a255-142b7543a38e
recipients=1 attachments=0 replyTo=null

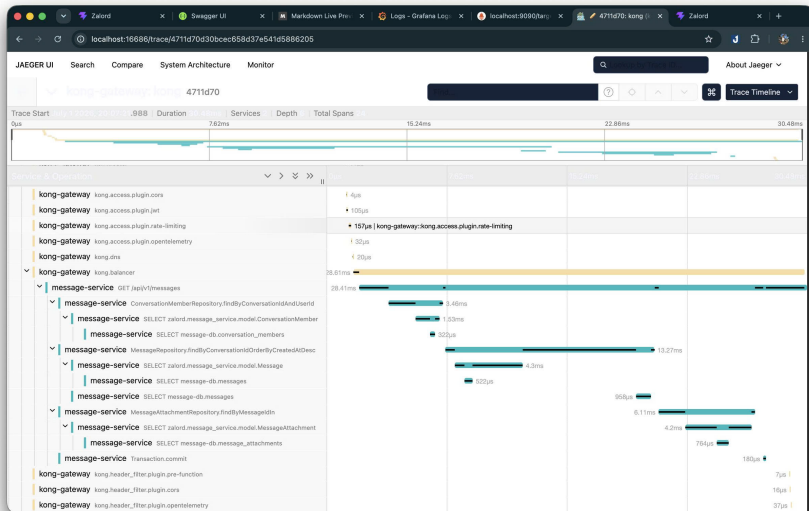
> 2026-07-01 20:00:07.324 2026-07-01T13:00:07.324Z INFO 1 --- [message-service] [nio-8883-exec-2] z.m.service.lmp1.MessageServiceImpl : Message sen
t id=cf031deb-d612-44ab-b1a2-e3355db21735 conv1cd3e397-48e8-42fc-9838-8db7fdd7c7ed sender=c84a1965-ddab-45ea-a255-142b7543a38e
recipients=1 attachments=0 replyTo=null

> 2026-07-01 20:00:07.329 2026-07-01T13:00:07.329Z INFO 1 --- [message-service] [nio-8883-exec-3] z.m.service.lmp1.MessageServiceImpl : Message sen
t id=65ca8ff6-d8cc-481e-b4de-77eb4964113a conv1cd3e397-48e8-42fc-9838-8db7fdd7c7ed sender=c84a1965-ddab-45ea-a255-142b7543a38e
recipients=1 attachments=0 replyTo=null
```

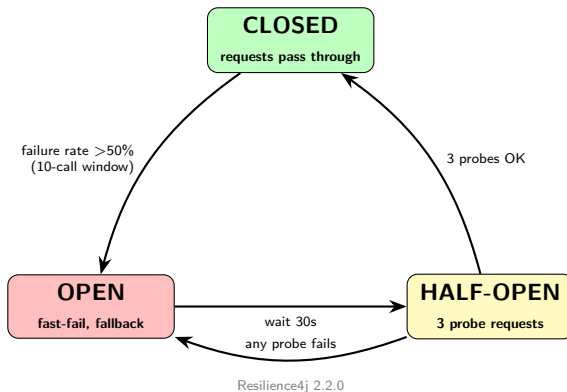

OpenTelemetry & Jaeger

- Sử dụng **OpenTelemetry (OTel)** để inject trace-id qua các HTTP header và gRPC metadata.
- **Luồng:** API Gateway → Microservice A → gRPC → Microservice B → RabbitMQ/Kafka → Consumer C.
- Dữ liệu được đẩy về **Jaeger** UI để trực quan hóa toàn bộ chuỗi request.

Distributed Tracing Flow — Jaeger UI



Circuit Breaker Diagram (Req 14)



Timeout, Circuit Breaker & Recovery (Req 14)

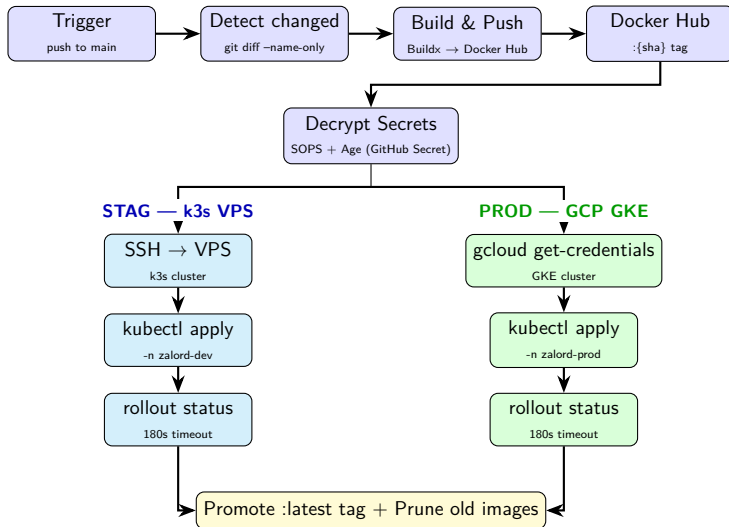
Implementation

- **auth-service** → user-service gRPC:
 - `@CircuitBreaker(name="userGrpc")`
 - gRPC deadline: **5 giây**
 - Fallback: `createProfileFallback()`
- **message-service** → media-service gRPC:
 - `@CircuitBreaker(name="mediaGrpc")`
 - gRPC deadline: **3 giây** (hot path)
 - Fallback: `validateFallback()`

Go services — Error Handling

- **PermanentError**: drop message (ack), log error
- **Transient error**: nack + requeue → RabbitMQ retry
- Frontend WebSocket: exponential backoff reconnect (cap 15s)

GitHub Actions Pipeline (Req 15)



GitHub Actions Pipeline (Req 15)

The screenshot shows a GitHub Actions workflow run for the repository 'thin0704hcm / zalord'. The workflow is named 'Deploy Production OKE' and is currently running at job #47. The status is 'succeeded'.

The workflow steps are as follows:

- Set up job
- Checkout
- Validate required secrets
 - Download CI image metadata
- Load CI image metadata
 - Download CI image manifest
- Load CI image manifest
- Detect changed services
- Set up kubectl
- Install deploy tools
- Configure SOPS age key
- Docker Hub login
- Check image existence
- Configure OKE kubeconfig
- Build deploy bundle
- Apply manifests
- Apply canary images

The workflow file is located at `.github/workflows/deploy.yml`.

Versioning & Rollback Strategy (Req 15)

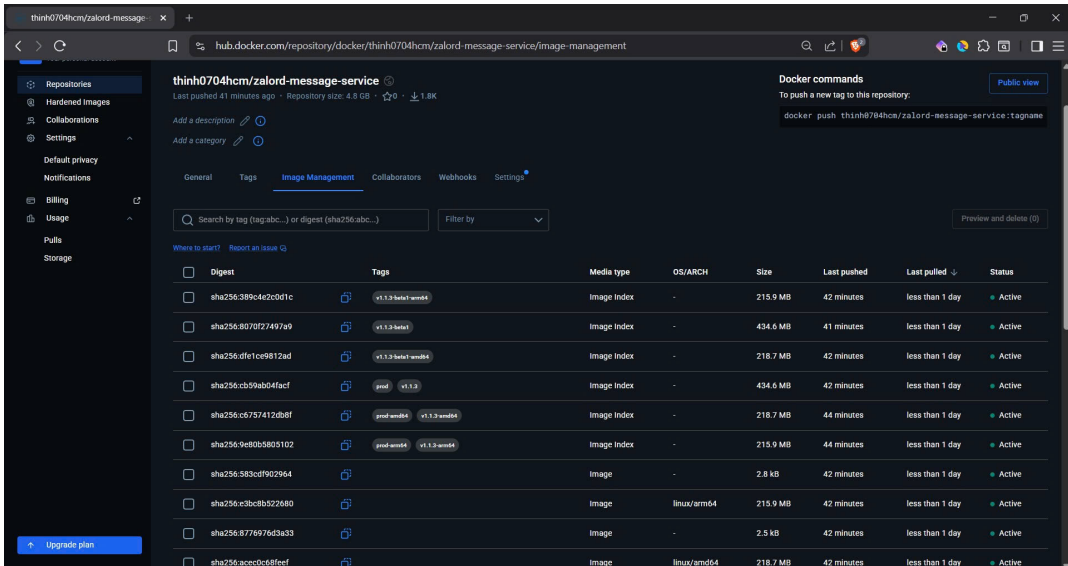
Image Versioning

- Mỗi image tagged với **Git SHA**:
 - `docker.io/{user}/zalord-auth:{sha}`
 - `docker.io/{user}/zalord-chat:{sha}`
 - ...
- Sau khi deploy thành công: promote `:latest` tag
- **Selective build**: chỉ rebuild service có file thay đổi → tiết kiệm thời gian CI
- Không có image nào bị overwrite — SHA tag immutable

Rollback

- K8s giữ revision history của Deployment
- `kubectl rollout undo ...` → về image SHA của commit trước
- `kubectl rollout status` trong pipeline → fail fast (180s) nếu pods không healthy

Versioning & Rollback Strategy (Req 15)



The screenshot shows the Docker Hub repository page for `thinh0704hcm/zalord-message-service`. The repository is public, last pushed 41 minutes ago, and has a size of 4.8 GB. The **Image Management** tab is selected, displaying a table of image versions.

Repository Details:

- Repository: `thinh0704hcm/zalord-message-service`
- Last pushed: 41 minutes ago
- Repository size: 4.8 GB
- Stars: 0
- Downloads: 1.8K

Docker commands:

```
docker push thinh0704hcm/zalord-message-service:tagname
```

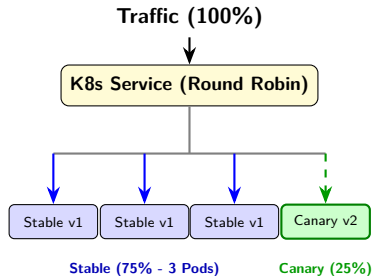
Image Management Table:

Digest	Tags	Media type	OS/ARCH	Size	Last pushed	Last pulled	Status
sha256:389c4e2c0d1c	v1.1.3-beta1-arm64	Image Index	-	215.9 MB	42 minutes	less than 1 day	Active
sha256:80f0f27497a9	v1.1.3-beta1	Image Index	-	434.6 MB	41 minutes	less than 1 day	Active
sha256:dfe1ce9812ad	v1.1.3-beta1-arm64	Image Index	-	218.7 MB	42 minutes	less than 1 day	Active
sha256:cb59ab04facf	prod v1.1.3	Image Index	-	434.6 MB	42 minutes	less than 1 day	Active
sha256:c6757412db8f	prod-arm64 v1.1.3-arm64	Image Index	-	218.7 MB	44 minutes	less than 1 day	Active
sha256:9e80b5805102	prod-arm64 v1.1.3-arm64	Image Index	-	215.9 MB	44 minutes	less than 1 day	Active
sha256:583cdf902964		Image	-	2.8 kB	42 minutes	less than 1 day	Active
sha256:c3bc8b522680		Image	linux/arm64	215.9 MB	42 minutes	less than 1 day	Active
sha256:8776976d3a33		Image	-	2.5 kB	42 minutes	less than 1 day	Active
sha256:ace0c68feef		Image	linux/arm64	218.7 MB	42 minutes	less than 1 day	Active

Canary Deployment

Chiến lược Canary

- Khi có tính năng rủi ro cao, ta có thể triển khai **Canary release**.
- Chạy song song 2 bản Deployment (stable và canary) chia sẻ chung cùng 1 Label Selector.
- K8s Service sẽ phân bổ traffic **round-robin** đều cho tất cả các Pod.
- Tỷ lệ chia traffic phụ thuộc vào tỷ lệ số lượng Pod (VD: 3 Stable Pods + 1 Canary Pod $\approx$ 75%/25%).



Tổng kết (1/2) — Đã triển khai

Kiến trúc & Backend

- ✓ DB-per-service (6 PostgreSQL + Redis + MinIO)
- ✓ Transactional Outbox (auth, message services)
- ✓ CQRS (message-service InboxProjector)
- ✓ Event-driven (RabbitMQ/Kafka switchable)
- ✓ gRPC sync + Circuit Breaker (Resilience4j)
- ✓ JWT HS256 auth (không cần OAuth2/Keycloak)

Hạ tầng & Vận hành

- ✓ Kong API Gateway (JWT, rate-limit, CORS, Lua)
- ✓ ConfigMap + SOPS-encrypted Secrets
- ✓ Multi-stage Dockerfile, Docker Compose
- ✓ Kubernetes (k3s, Kustomize)
- ✓ CI/CD GitHub Actions → Docker Hub → GKE / k3s
- ✓ Prometheus, Grafana, Loki (Logging)
- ✓ OpenTelemetry + Jaeger (Tracing)
- ✓ Swagger/OpenAPI aggregation

Chưa triển khai / Không phù hợp

- Saga Pattern — không có distributed transaction phức tạp
- OAuth2/Keycloak — không phù hợp: closed system

Hướng phát triển tương lai

- Mở rộng tính năng gọi thoại (Voice Call) / gọi video (Video Call).
- Triển khai Service Mesh (Istio) khi số lượng service tăng lên.

**Cảm ơn thầy và các bạn
đã lắng nghe!**